Topic 11: Pointers and Memory

10.1 What is Memory?

- RAM is like a very long street of numbered houses — each house holds one byte
- Every variable in your program lives at a specific address in RAM
- The address of variable `x` is written as `&x`
- Understanding memory is what separates C++ from higher-level languages

10.2 What is a Pointer?

- A pointer is a variable that stores the memory address of another variable
- `int* p = &x;` — p holds the address of x
- `*p` dereferences the pointer — gives the value at that address
- `int* p;` declares p as a pointer to int — the `*` is part of the type
- Null pointer: `int* p = nullptr;` — pointer that points to nothing (safe default)

10.3 Using Pointers

```
int x = 10;
int* p = &x;
cout << p << endl;   // prints address, e.g. 0x7fffd4
cout << *p << endl;  // prints 10
*p = 20;             // changes x to 20 through the pointer
cout << x << endl;   // prints 20
```

10.4 Pointers and Arrays

- Array name is a pointer to the first element: `int arr[] = {1, 2, 3}; int* p = arr;`
- `*(p + 1)` accesses the second element (same as `arr[1]`)
- Pointer arithmetic: `p + 1` moves to the next int-sized address (4 bytes forward)

10.5 Pass by Pointer (Modifying via Function)

```
void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

```
int x = 5, y = 10;
swap(&x, &y);  // x=10, y=5 after
```

10.6 Pointers vs References

Pointer	Reference
Can be null	Cannot be null
Can be reassigned	Always refers to same variable
`int* p = &x;`	`int& r = x;`
Explicit dereference `*p`	Transparent — use like variable

- References are simpler and safer for most use cases; pointers give more control

10.7 Memory Diagram

- When `int x = 5;` is written, the compiler reserves 4 bytes at, say, address 0x100 and stores 5
- `int* p = &x;` reserves 8 bytes (pointer size on 64-bit) at address 0x108 and stores 0x100
- `*p` reads the value at address 0x100 → 5
